Documentación del backend

Fase 1: API CRUD de posts con Go y Gin

Proyecto: crud-go-react (curso de Udemy: Go + React + Gin)

Repositorio: https://github.com/Gariasa83/crud-go-react

Fecha: 28 de octubre de 2026

Estado: backend terminado y probado en Insomnia; primer commit subido a GitHub

Contenido

1. Objetivo y resultado de la fase
2. Tecnologías y versiones
3. Preparación del entorno
4. Estructura del proyecto
5. Construcción del backend, paso a paso
6. Endpoints de la API
7. Cómo ejecutar el servidor
8. Pruebas con Insomnia
9. Errores encontrados y cómo se corrigieron
10. Subida del primer commit a GitHub
11. Pendientes y mejoras

1. Objetivo y resultado de la fase

Construir una API REST en Go con el framework Gin que permita administrar posts (crear, listar, consultar, actualizar y borrar), separada en capas y lista para conectarla con un frontend en React.

Resultado: una API con cinco rutas bajo `/posts`, respuestas JSON con formato uniforme, CORS habilitado, recarga automática en desarrollo con Air y pruebas manuales hechas en Insomnia.

Todas las respuestas tienen la misma forma:

```
{
  "ok": true,
  "data": [ ... ],
  "message": "lista de posts"
}
```

2. Tecnologías y versiones

Herramienta	Versión	Para qué se usa
Go	1.27.3	Lenguaje del backend
Gin (`github.com/gin-gonic/gin`)	v1.12.0	Framework web: rutas, JSON y validación
uuid (`github.com/google/uuid`)	v1.6.0	Generar ids únicos para los posts
cors (`github.com/gin-contrib/cors`)	v1.7.9	Permitir peticiones desde el frontend en otro puerto
Air (`github.com/air-verse/air`)	v1.67.4	Reiniciar el servidor al guardar cambios
Insomnia	-	Probar los endpoints manualmente
Git y GitHub CLI (`gh`)	git 2.53.0	Control de versiones y publicación

3. Preparación del entorno

3.1 Crear el módulo de Go

Dentro de la carpeta `backend` se inicializa el módulo. El nombre `backend` es el que se usa después en los imports internos (`"backend/models"` , `"backend/helpers"` , etc.).

```
cd ~/Documentos/GO/CRUD/backend
go mod init backend
```

3.2 Instalar las dependencias

```
go get -u github.com/gin-gonic/gin
go get github.com/google/uuid
go get github.com/gin-contrib/cors
```

> **Importante:** `go mod tidy` elimina del `go.mod` las dependencias que ningún archivo importa. Si se ejecuta antes de escribir el `import` correspondiente, la dependencia desaparece y aparece el error *no required module provides package*. Orden correcto: primero el `import` en el código y luego `go mod tidy` .

3.3 Instalar Air (recarga automática)

El repositorio original (`cosmtrek/air`) se mudó de organización; hoy se instala desde `air-verse/air` . Es la misma herramienta.

```
go install github.com/air-verse/air@latest
```

Air queda en `~/go/bin` . Para poder llamarlo como `air` hay que añadir esa carpeta al PATH, agregando esta línea al final de `~/.bashrc` y abriendo una terminal nueva (o ejecutando

`source ~/.bashrc`):

```
export PATH="$PATH:$HOME/go/bin"
```

Dentro de `backend`, una sola vez, se genera su configuración (`air.toml`):

```
air init
```

4. Estructura del proyecto

El backend sigue una separación por capas: cada carpeta tiene una única responsabilidad.

```
CRUD/
├── backend/
│   ├── main.go                 punto de entrada: rutas y CORS
│   ├── go.mod / go.sum         dependencias
│   ├── air.toml                configuración de Air
│   ├── models/
│   │   └── post.go             estructura PostData
│   ├── database/
│   │   └── data.go             datos en memoria (lista de posts)
│   ├── helpers/
│   │   └── response.go         función Response (respuesta JSON uniforme)
│   └── controllers/
│       └── post-controller.go  lógica de cada endpoint
└── frontend/                   (React + Vite, siguiente fase)
```

5. Construcción del backend, paso a paso

Paso 1. Modelo (`models/post.go`)

Define la forma de un post. Los *tags* entre comillas invertidas (acento grave) indican el nombre del campo en el JSON. `binding:"required"` hace que Gin rechace con error 400 los posts sin título.

```
package models

// CREAR EL MODELO
type PostData struct {
        ID          string `json:"id"`
        Title       string `json:"title" binding:"required"`
        Description string `json:"description"`
        ImgUrl      string `json:"imgUrl"`
}
```

Paso 2. Datos en memoria (`database/data.go`)

Una lista global con posts de ejemplo. Los ids se generan con `uuid.New().String()`. Los textos largos van entre comillas invertidas, que sí permiten varias líneas. En esta fase los datos viven en memoria: al reiniciar el servidor se pierden los cambios y los ids de los posts iniciales cambian.

```
package database

import (
        "backend/models"

        "github.com/google/uuid"
)

var Data = []models.PostData{
        {
                ID:          uuid.New().String(),
                Title:       "Mejor album de la banda In Flames",
                Description: `Subterranean es un EP de la banda sueca ...`,
                ImgUrl:      "https://upload.wikimedia.org/.../In_Flames_-
_Subterranean.jpg",
        },
        {
                ID:          uuid.New().String(),
                Title:       "Mejor album de la banda Cradle of Filth",
                Description: `Dusk and Her Embrace es el segundo álbum ...`,
                ImgUrl:      "https://m.media-
amazon.com/images/I/51dFPixpvzL._UF1000,1000_QL80_.jpg",
        },
}
```

Las imágenes deben ser enlaces directos a un archivo (.jpg, .png), no páginas; de lo contrario no se mostrarán en un `<img>` del frontend.

Paso 3. Respuesta uniforme (`helpers/response.go`)

Una función que evita repetir el mismo `c.JSON(...)` en cada controlador y garantiza que todas las respuestas tengan `ok`, `data` y `message`.

```
package helpers

import "github.com/gin-gonic/gin"

// RESPONSE
func Response(c *gin.Context, statusCode int, ok bool, data interface{}, message string) {
        c.JSON(statusCode, gin.H{
                "ok":      ok,
                "data":    data,
                "message": message,
        })
}
```

Paso 4. Controladores (`controllers/post-controller.go`)

Una función por operación del CRUD. Puntos clave:

- `c.Param("id")` (sin *s*) lee el parámetro de la ruta.
- `c.ShouldBindJSON(&post)` convierte el cuerpo JSON en la estructura y valida los *tags*.
- El id lo asigna siempre el servidor con uuid, nunca el cliente.
- Un post inexistente responde `404 Not Found`; un JSON inválido, `400 Bad Request`; una creación, `201 Created`.

```
package controllers

import (
	"backend/database"
	"backend/helpers"
	"backend/models"
	"net/http"

	"github.com/google/uuid"

	"github.com/gin-gonic/gin"
)

// LISTAR TODA LA DATA
func GetAllData(c *gin.Context) {
	helpers.Response(c, http.StatusOK, true, database.Data, "lista de posts")
}

// OBTENER UN POST POR ID
func GetPostByID(c *gin.Context) {
	id := c.Param("id")
	for _, post := range database.Data {
		if post.ID == id {
			helpers.Response(c, http.StatusOK, true, post, "post encontrado")
			return
		}
	}
	helpers.Response(c, http.StatusNotFound, false, "", "post no encontrado")
}

// CREAR UN POST
func CreatePost(c *gin.Context) {
	var post models.PostData
	if err := c.ShouldBindJSON(&post); err != nil {
		helpers.Response(c, http.StatusBadRequest, false, "", err.Error())
		return
	}

	post.ID = uuid.New().String()
	database.Data = append(database.Data, post)

	helpers.Response(c, http.StatusCreated, true, post, "post creado")
}

// ACTUALIZAR UN POST
func UpdatePost(c *gin.Context) {
	id := c.Param("id")
	var updatePost models.PostData

	if err := c.ShouldBindJSON(&updatePost); err != nil {
		helpers.Response(c, http.StatusBadRequest, false, "", err.Error())
		return
	}
	for i, post := range database.Data {
		if post.ID == id {
			updatePost.ID = database.Data[i].ID
			database.Data[i] = updatePost
			helpers.Response(c, http.StatusOK, true, "", "post actualizado")
			return
		}
	}
```

```
        helpers.Response(c, http.StatusNotFound, false, "", "post no encontrado")
}

// ELIMINAR UN POST
func DeletePost(c *gin.Context) {
        id := c.Param("id")

        for i, post := range database.Data {
                if post.ID == id {
                        database.Data = append(database.Data[:i], database.Data[i+1:]...)
                        helpers.Response(c, http.StatusOK, true, "", "post eliminado")
                        return
                }
        }
        helpers.Response(c, http.StatusNotFound, false, "", "post no encontrado")
}
```

Paso 5. Rutas y CORS (`main.go`)

Crea el router, registra el middleware de CORS **antes** de las rutas (Gin solo lo aplica a las rutas registradas después) y enlaza cada ruta con su controlador. El servidor escucha en el puerto 8080.

```
package main

import (
        "backend/controllers"

        "github.com/gin-contrib/cors"
        "github.com/gin-gonic/gin"
)

func main() {
        router := gin.Default()

        // CONFIGURAR CORS
        config := cors.DefaultConfig()
        config.AllowAllOrigins = true

        corsMiddleware := cors.New(config)

        router.Use(corsMiddleware)

        router.GET("/posts", controllers.GetAllData)
        router.GET("/posts/:id", controllers.GetPostByID)
        router.POST("/posts", controllers.CreatePost)
        router.PUT("/posts/:id", controllers.UpdatePost)
        router.DELETE("/posts/:id", controllers.DeletePost)
        router.Run(":8080")
}
```

CORS: el navegador bloquea las llamadas de React (puerto 5173) a la API (puerto 8080) si la API no lo permite. `AllowAllOrigins = true` es cómodo en desarrollo; en producción conviene limitarlo a la dirección del frontend.

6. Endpoints de la API

Método	Ruta	Descripción	Éxito	Errores
GET	`/posts`	Lista todos los posts	200	-
GET	`/posts/:id`	Devuelve un post por su id	200	404
POST	`/posts`	Crea un post (el id lo genera el servidor)	201	400
PUT	`/posts/:id`	Reemplaza los datos de un post y conserva su id	200	400, 404
DELETE	`/posts/:id`	Elimina un post	200	404

7. Cómo ejecutar el servidor

Siempre desde la carpeta `backend`, que es donde está el `go.mod`:

```
cd ~/Documentos/GO/CRUD/backend
air               # con recarga automática al guardar
# o bien:
go run main.go    # sin recarga automática
```

Cuando arranca bien, la terminal muestra `Listening and serving HTTP on :8080`. Los avisos `[WARNING]` sobre el modo debug y los proxies son normales en desarrollo.

> **No ejecutar desde `CRUD`:** Go responde *go.mod file not found*, Air no compila y termina con *tmp/main: not found*.

8. Pruebas con Insomnia

8.1 Crear la colección

1. En Insomnia, crear un proyecto (por ejemplo **CRUD Go React**).
2. Dentro del proyecto pulsar **Create** y elegir **API Collection**; ponerle un nombre (por ejemplo **CRUD API**).
3. Crear una petición por cada endpoint con **New HTTP Request**.
4. Opcional: en **Base Environment** guardar `{ "base_url": "http://localhost:8080" }` y usar `{{ _.base_url }}/posts` en las URLs.

8.2 Peticiones

Petición	Método y URL	Cuerpo (Body > JSON)	Resultado esperado
Listar posts	GET `http://localhost:8080/posts`	-	200 con `data` y la lista de posts
Obtener un post	GET `.../posts/<id>`	-	200 con el post; 404 si el id no existe
Crear post	POST `.../posts`	título, descripción e imagen (ver abajo)	201 con el post y su nuevo id
Actualizar post	PUT `.../posts/<id>`	los campos nuevos	200 "post actualizado"
Eliminar post	DELETE `.../posts/<id>`	-	200 "post eliminado"; repetirlo da 404

Cuerpo de ejemplo para crear o actualizar (los nombres deben coincidir con los *tags* del modelo):

```
{
  "title": "Mi primer post",
  "description": "Descripción de prueba",
  "imgUrl": "https://ejemplo.com/foto.jpg"
}
```

Verificaciones recomendadas: enviar un POST sin `title` debe responder 400; después de crear, editar o borrar, volver a ejecutar **Listar posts** para confirmar el cambio.

> Los ids de los posts iniciales cambian cada vez que se reinicia el servidor (incluido el reinicio automático de Air). Hay que copiar un id nuevo con GET `/posts` antes de probar GET, PUT o DELETE por id.

Si hay problemas: *Connection refused* significa que el servidor no está corriendo; un 404 indica que la ruta o el método no coincide con `main.go`; un 400 indica JSON mal formado o título vacío.

9. Errores encontrados y cómo se corrigieron

Error	Causa	Solución
`expected ';', found 'json:"id"'`	Tags del struct escritos con comillas simples `'...'`	Usar comillas invertidas (acento grave) en los tags
`string literal not terminated`	Texto de varias líneas entre comillas dobles	Usar comillas invertidas o escribirlo en una sola línea
`undefined: uuid`	Paquete instalado pero no importado	Añadir `"github.com/google/uuid"` al bloque `import`
`no required module provides package ...uuid`	`go mod tidy` había quitado la dependencia por no estar importada	Importar primero y luego ejecutar `go get` y `go mod tidy`
`cannot call c.Params`	`Params` es un campo (lista); el método se llama `Param`	Escribir `c.Param("id")`
Respuesta 302 al no encontrar un post	Se usó `http.StatusFound` (redirección)	Usar `http.StatusNotFound` (404)
`undefined: i` en `data[i] = ...`	El `for` descartaba el índice con `_`	`for i, post := range ...`
`label data declared and not used`	`data:append(...)` con dos puntos, sin `=`	`data = append(...)`
`declared and not used: config`	Se creó la configuración de CORS pero no se registró	Usar `router.Use(cors.New(config))` antes de las rutas
`go.mod file not found` al ejecutar Air	Se ejecutó desde `CRUD` y no desde `backend`	`cd backend` y luego `air`

10. Subida del primer commit a GitHub

Se creó un único repositorio para todo el proyecto (carpetas `backend` y `frontend`) y se subió primero solo el backend.

1. Configurar la identidad de git (una sola vez):

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu-correo@ejemplo.com"
```

2. Iniciar sesión en GitHub: `gh auth login` (se comprueba con `gh auth status`).
3. Crear el repositorio local en la carpeta `CRUD` con la rama principal `main`:

```
git init -b main
```

4. Crear un `.gitignore` para no subir archivos generados ni sobrantes:

```
node_modules/
tmp/
backend/backend
.env
claude resume*.txt
```

5. Añadir solo lo que corresponde y revisar:

```
git add backend .gitignore
git status
```

6. Hacer el commit:

```
git commit -m "feat(backend): API CRUD de posts con Go y Gin"
```

7. Crear el repositorio en GitHub, conectarlo y subir `main` en un solo paso:

```
gh repo create crud-go-react --public --source=. --remote=origin --push
```

> **Resultado:** commit `031ba7d` con 9 archivos publicado en **https://github.com/Gariasa83/crud-go-react** (rama `main`).

No fue necesario crear una rama para el primer commit. Para cambios posteriores el flujo es `git add`, `git commit` y `git push`; las ramas (por ejemplo `feature/base-de-datos`) son útiles para trabajar una función nueva sin tocar `main`.

11. Pendientes y mejoras

- Reemplazar los datos en memoria por una base de datos real (PostgreSQL, SQLite o MongoDB).
- `UpdatePost` podría devolver el post actualizado en `data`, útil para el frontend.
- Limitar `AllowAllOrigins` a la dirección del frontend en producción.
- Corregir la ortografía en nombres y comentarios y añadir comentarios a `UpdatePost` y `DeletePost`.
- Añadir un `README.md` al repositorio y subir el frontend cuando se termine esa parte del curso.
- Considerar pruebas automáticas y despliegue para el portafolio.